

Obtenção de dados de rastreamento ocular com webcam utilizando Inteligência Artificial para avaliação da experiência do usuário

Bruno M. Ribas

brunoribas68@gmail.com

Setor de Educação Profissional e Tecnológica,
Universidade Federal do Paraná
Curitiba, Paraná, Brasil

Jaime Wojciechowski

jaimewo@ufpr.br

Setor de Educação Profissional e Tecnológica,
Universidade Federal do Paraná
Curitiba, Paraná, Brasil

RESUMO

O rastreamento ocular permite observar para onde os usuários olham ao interagir com um produto digital e é uma fonte valiosa de dados para avaliar a experiência do usuário (UX). Contudo, rastreadores dedicados são caros e exigem laboratório, o que limita seu uso. Este trabalho, apresentado como conclusão da Especialização em Inteligência Artificial Aplicada, desenvolve e avalia um protótipo de rastreamento ocular que utiliza apenas uma webcam comum. O sistema detecta o rosto e a íris com uma rede neural convolucional (MediaPipe Face Mesh), aprende por regressão regularizada o mapeamento entre a posição da íris e a tela a partir de uma calibração de nove pontos, suaviza o sinal com o filtro One Euro e segmenta fixações com o algoritmo I-DT, gerando mapas de calor, trajetórias do olhar e métricas por área de interesse. Em um experimento com [N] participantes, o sistema obteve acurácia média de [X]° de ângulo visual, precisão de [Y]° e perda de dados de [Z]%. **[Uma frase de conclusão: por exemplo, a precisão obtida é suficiente para identificar quais regiões de uma página atraem a atenção, mas não para análises em nível de palavra.]** O código é aberto e reprodutível.

ABSTRACT

Eye tracking reveals where users look while interacting with a digital product and is a valuable data source for user experience (UX) evaluation. However, dedicated eye trackers are expensive and require a laboratory, which limits their adoption. This work, the final paper of the Specialization in Applied Artificial Intelligence, develops and evaluates an eye-tracking prototype that relies solely on a commodity webcam. The system detects the face and iris with a convolutional neural network (MediaPipe Face Mesh), learns the mapping from iris position to screen coordinates through regularized regression on a nine-point calibration, smooths the signal with the One Euro filter, and segments fixations with the I-DT algorithm, producing heatmaps, scanpaths, and area-of-interest metrics. In an experiment with [N] participants, the system achieved a mean accuracy of [X]° of visual angle, a precision of [Y]°, and [Z]% data loss. **[Conclusion sentence.]** The code is open and reproducible.

CCS CONCEPTS

• Human-centered computing → HCI design and evaluation methods; • Computing methodologies → Computer vision.

KEYWORDS

rastreamento ocular, webcam, experiência do usuário, visão computacional, aprendizado de máquina

1 INTRODUÇÃO

A experiência do usuário (UX) é um aspecto crucial para determinar a qualidade de um produto ou serviço. Diversos indicadores são empregados para compreendê-la, incluindo questionários de satisfação, perguntas sobre aspectos específicos do produto e medidas de tempo, como o tempo necessário para concluir tarefas e o tempo de resposta do sistema. Apesar da variedade de indicadores, há esforços para combiná-los, como em Strohl et al. [34], que usam rastreamento ocular para aprimorar formulários.

Entre esses indicadores, o rastreamento ocular (*eye tracking*) tem se mostrado uma ferramenta promissora, especialmente na avaliação de produtos digitais [36, 3]: ele revela o que o usuário de fato observa, e não apenas o que ele relata ter observado. Trabalhar com dados oculares, porém, é desafiador. A qualidade dos dados varia com a calibração do equipamento, a movimentação do participante e características fisiológicas como a cor dos olhos, o que leva à perda de dados (*data loss*) [25, 11]. Além disso, rastreadores dedicados, baseados em iluminação infravermelha, têm custo elevado e normalmente exigem um laboratório [13].

Avanços recentes em visão computacional e aprendizado de máquina tornaram possível estimar o olhar a partir de uma webcam comum [26, 19, 28]. Essa abordagem tem precisão inferior à dos equipamentos dedicados, mas custo praticamente nulo, o que poderia democratizar testes de UX com rastreamento ocular. Resta saber se a qualidade obtida é suficiente para responder às perguntas que interessam a quem projeta interfaces.

1.1 Objetivos

O objetivo geral deste trabalho é desenvolver e avaliar um protótipo de rastreamento ocular baseado em webcam e em técnicas de Inteligência Artificial, capaz de gerar dados úteis para a avaliação da experiência do usuário. Os objetivos específicos são:

- (1) implementar um *pipeline* que detecte o rosto e a íris, estime o ponto observado na tela e identifique fixações;
- (2) medir a qualidade dos dados obtidos (acurácia, precisão e perda de dados) segundo métricas consolidadas na literatura [14];
- (3) comparar modelos de calibração (regressão linear e polinomial, com e sem informação da pose da cabeça);

- (4) demonstrar o uso dos dados em uma tarefa de UX, por meio de mapas de calor e métricas por área de interesse.

O restante do artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica e os trabalhos relacionados; a Seção 3 descreve o protótipo e o protocolo experimental; a Seção 4 apresenta e discute os resultados; e a Seção 5 traz as considerações finais.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Experiência do Usuário (UX)

A Experiência do Usuário pode ser descrita como o conjunto de percepções e respostas de uma pessoa ao usar um produto ou serviço, e pode ser avaliada a partir de diversas interações do usuário. Morville [22] propõe que uma boa experiência reúne sete facetas: o produto deve ser útil, utilizável (fácil de usar), desejável, localizável (o usuário encontra o que procura), acessível (inclusive a pessoas com deficiência), confiável e valioso. O rastreamento ocular contribui especialmente para avaliar as facetas *utilizável* e *localizável*, pois mostra se o usuário encontra rapidamente os elementos de que precisa.

Nielsen [23] apresenta uma abordagem sistemática para melhorar a usabilidade de sistemas interativos, enfatizando produtos fáceis de aprender, eficientes de usar e que proporcionem satisfação. Norman [24], por sua vez, introduz conceitos fundamentais para o design centrado no usuário:

Affordances e feedback as possibilidades de ação que um objeto comunica intuitivamente devem ser claras, reduzindo a necessidade de instruções; o sistema deve informar imediatamente o resultado das ações.

Mapeamento a relação entre os controles e seus efeitos deve corresponder às expectativas naturais do usuário.

Design emocional além de funcional, o design deve evocar uma resposta emocional positiva, considerando aspectos estéticos e psicológicos.

Erros de design muitos erros de uso não são culpa do usuário, mas de um design com pouca visibilidade ou *feedback* inadequado.

Problemas de affordance e de visibilidade descritos por Norman costumam aparecer diretamente nos dados oculares: um botão que não “parece clicável” recebe poucas fixações, ou é encontrado só depois de uma longa busca visual.

2.2 Rastreamento Ocular

O rastreamento ocular consiste em medir para onde os olhos estão direcionados ao longo do tempo, a partir da posição e do movimento da pupila [13, 8]. Ao examinar os movimentos oculares, é possível obter percepções sobre os processos de atenção das pessoas e aprender o que elas consideram importante, interessante ou confuso [3].

Dois eventos oculares são centrais para a análise [31]: as *fixações*, períodos em que o olhar permanece relativamente parado sobre uma região e durante os quais a informação visual é efetivamente processada; e as *sacadas* (*saccades*), movimentos rápidos em que o olhar salta de um ponto a outro. Existem outros movimentos, como a perseguição suave e os micro-sacádicos [8], mas fixações e sacadas

bastam para a maioria das análises de UX. Identificar automaticamente esses eventos no sinal bruto não é trivial: Andersson et al. [2] compararam dez algoritmos de detecção de eventos e mostraram que eles divergem substancialmente entre si, e Braunagel et al. [5] argumentam que a classificação precisa se adaptar ao contexto de uso.

A partir das fixações calculam-se as métricas usuais de UX [13, 3]: o tempo até a primeira fixação (*time to first fixation*, TTFF) em uma área de interesse (AOI), que indica quão rapidamente um elemento é encontrado; o número de fixações e o tempo de permanência (*dwell time*) em cada AOI, que indicam interesse ou dificuldade; os mapas de calor, que agregam a atenção de vários participantes; e a trajetória do olhar (*scanpath*), que mostra a ordem de exploração.

2.2.1 Qualidade dos dados. A confiabilidade dessas métricas depende da qualidade dos dados, descrita principalmente por três medidas [14]: a *acurácia*, distância média entre o ponto estimado e o ponto realmente observado; a *precisão*, a dispersão das amostras quando o olhar está parado; e a *perda de dados*, a proporção de amostras em que o olhar não pôde ser estimado. Essas medidas são geralmente expressas em graus de ângulo visual, o que as torna comparáveis entre telas e distâncias diferentes. Fatores como o método de calibração e a fisiologia do olho afetam todas elas [25], assim como a cor dos olhos, o posicionamento e o movimento da cabeça [11]. No estudo longitudinal YOUTH, que realizou mais de 1.500 gravações com crianças, Hessels e Hooge [12] mostraram que a qualidade dos dados depende inclusive do operador que conduz a sessão, mesmo após treinamento. Hooge et al. [15] mostraram ainda que, em alguns casos, um único olho fornece dados mais acurados do que a média dos dois.

2.2.2 Rastreamento com webcam. Rastreadores comerciais usam iluminação infravermelha e câmeras de alta frequência, e atingem acurácias inferiores a 1° em condições controladas [13]. O custo desses equipamentos motivou abordagens remotas que utilizam câmeras comuns com apoio de técnicas de IA. Há duas famílias principais: as baseadas em *características*, que localizam pontos do olho (pupila, íris, cantos) e mapeiam sua posição para a tela por meio de uma calibração, como o WebGazer [26]; e as baseadas em *aparência*, que treinam redes neurais para estimar a direção do olhar diretamente a partir da imagem do olho ou do rosto [19, 37, 28, 27]. Este trabalho segue a primeira abordagem, que exige menos dados e menos poder computacional.

2.3 Técnicas de Inteligência Artificial e Visão Computacional

2.3.1 Histograma de Gradientes Orientados (HOG). O método de extração de características HOG foi introduzido por Dalal e Triggs [7] como parte de um algoritmo de detecção de pedestres. Ele extrai informações sobre a orientação das arestas de uma imagem, calculadas por operadores de gradiente como o de Sobel e Feldman [32]. O HOG divide a imagem em pequenas regiões, chamadas células, e calcula um histograma de gradientes orientados para cada uma. Esses histogramas são normalizados em blocos para aumentar a robustez a variações de iluminação e contraste. Combinado a um classificador linear, o HOG é um detector de rostos eficaz e é o detector padrão da biblioteca dlib [18].

2.3.2 Árvore de regressão para alinhamento facial. Árvores de regressão são modelos de aprendizado de máquina que preveem valores contínuos dividindo os dados de entrada em subconjuntos cada vez mais simples, com base em regras de decisão aprendidas no treinamento; cada folha armazena um valor de saída. Kazemi e Sullivan [17] utilizaram um conjunto (*ensemble*) de árvores de regressão em cascata para alinhar rostos em cerca de um milissegundo, prevendo a posição de 68 pontos de referência faciais (*landmarks*). O modelo de 68 pontos distribuído com a *dlib* foi treinado no conjunto de dados iBUG 300-W [30].

2.3.3 Redes neurais convolucionais e o MediaPipe Face Mesh. Redes neurais convolucionais (CNNs) aprendem diretamente da imagem as características relevantes para a tarefa. O MediaPipe [20] disponibiliza o Face Mesh, uma CNN leve que estima 468 pontos tridimensionais da superfície do rosto em tempo real, mesmo em dispositivos móveis [16]. Uma extensão do modelo estima mais 10 pontos que descrevem o contorno e o centro de cada íris [1]. Diferentemente dos 68 pontos da *dlib*, que contornam a pálpebra mas não localizam a íris, esses pontos permitem estimar a direção do olhar diretamente.

Para fins de comparação, o OpenCV oferece também o detector clássico de Viola-Jones [35], uma cascata de classificadores sobre características de Haar, que localiza rostos e olhos sem redes neurais.

2.3.4 Regressão para calibração. Em rastreadores baseados em características, uma etapa de calibração aprende uma função que mapeia as características do olho para coordenadas da tela: o participante observa alvos em posições conhecidas, e um modelo de regressão é ajustado a esses pares. Mapeamentos polinomiais de segunda ordem são comuns na literatura [13], e o WebGazer utiliza regressão *ridge* [26], cuja penalização sobre os coeficientes reduz o sobreajuste quando há poucos pontos de calibração.

2.4 Trabalhos Relacionados

Bojko [3] discute como planejar e analisar testes de UX com rastreamento ocular, usando como exemplo diversos sites. Um deles é a página inicial da Starbucks (Figura 1), cujos mapas de calor mostram a atividade combinada do olhar de usuários procurando o localizador de lojas. O trabalho mostra que o rastreamento ocular traz informações úteis sobre a experiência do usuário, mas que a pesquisa precisa ser cuidadosamente planejada.

Pretorius et al. [29] compararam usuários experientes e inexperientes em testes de usabilidade e mostraram que o rastreamento ocular revela problemas que não aparecem nos métodos tradicionais. Modi e Singh [21] estudaram o olhar em uma rede social usando apenas uma câmera: o rosto é detectado, as regiões dos olhos são recortadas e uma CNN classifica a direção do olhar, permitindo gerar mapas de calor. O WebGazer [26] estima o olhar no próprio navegador, recalibrando-se continuamente a partir dos cliques do usuário. Entre as abordagens baseadas em aparência, Krafka et al. [19] treinaram uma CNN com dados coletados de mais de 1.400 pessoas em celulares e *tablets*, Zhang et al. [37] mostraram a importância de normalizar a imagem do olho em relação à pose da cabeça, Park et al. [28] propuseram localizar pontos do olho com redes neurais para estimar o olhar em condições não controladas, e Park et al. [27] estimam o olhar a partir de vídeo de ponta a ponta.



Figura 1: Mapas de calor da busca pelo localizador de lojas na página da Starbucks. À esquerda, usuários ($n = 13$) viram uma versão em preto e branco da página; à direita ($n = 12$), a versão colorida. Cores mais quentes indicam maior porcentagem de usuários fixando o olhar na área. Fonte: Bojko [3].

Por fim, Fuhl et al. [9] utilizaram redes totalmente convolucionais para segmentar, gerar e reconstruir dados oculares brutos.

Este trabalho se diferencia por focar em um *pipeline* completo, simples e aberto, que vai da imagem da webcam até as métricas de UX, e por reportar a qualidade dos dados com as métricas padronizadas de Holmqvist et al. [14], o que muitas vezes falta em protótipos com webcam.

3 MATERIAIS E MÉTODOS

3.1 Visão geral

O protótipo foi desenvolvido em duas etapas. Uma primeira versão, exploratória, usou HOG e o modelo de 68 pontos da *dlib*. As limitações observadas motivaram uma segunda versão, baseada no MediaPipe, que constitui o *pipeline* avaliado neste trabalho (Figura 2). O código está disponível em <https://github.com/brunoriba/s68/eyetracking>.

3.2 Protótipo inicial: HOG e 68 pontos faciais

Na primeira versão, o rosto era detectado em tempo real com o detector HOG da *dlib* e, em seguida, os 68 pontos faciais eram estimados pelo modelo de árvores de regressão de Kazemi e Sullivan [17]. A partir dos pontos que contornam cada olho, a região ocular era recortada (Figura 3). Para localizar a pupila, a imagem era binarizada com um limiar ajustado manualmente, de modo que restassem apenas as regiões dos olhos (Figura 4), e o centro de cada região era marcado (Figura 5).

Essa versão confirmou que é possível localizar os olhos em tempo real com uma webcam, mas revelou três limitações: (i) os 68 pontos contornam as pálpebras, mas não localizam a íris, de modo que a pupila dependia da binarização; (ii) o limiar de binarização precisava ser reajustado a cada mudança de iluminação e era afetado pelos reflexos das lentes de óculos; e (iii) não havia calibração, portanto o sistema localizava o olho, mas não o ponto observado na tela. A segunda versão foi construída para superar esses pontos.

3.3 Pipeline final

3.3.1 Detecção e extração de características. Cada *frame* da webcam é processado pelo MediaPipe Face Mesh com o refinamento da íris [16, 1]. Para cada olho, sejam c o centro da íris (média dos seus

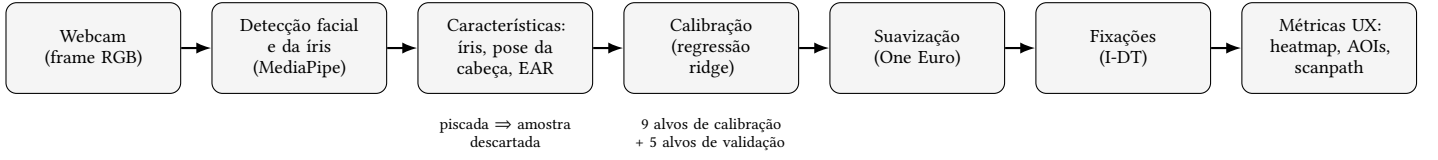


Figura 2: Pipeline do protótipo: da imagem da webcam às métricas de experiência do usuário.

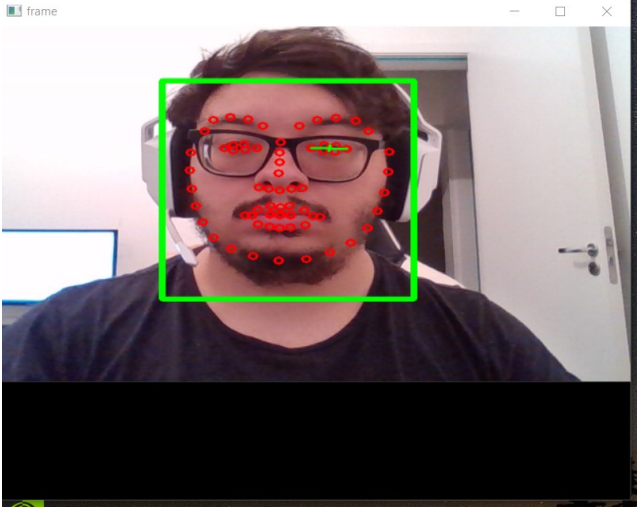


Figura 3: Protótipo inicial. O quadrado verde indica o rosto detectado pelo HOG, os pontos vermelhos são os 68 pontos faciais e o traço verde marca o olho direito.

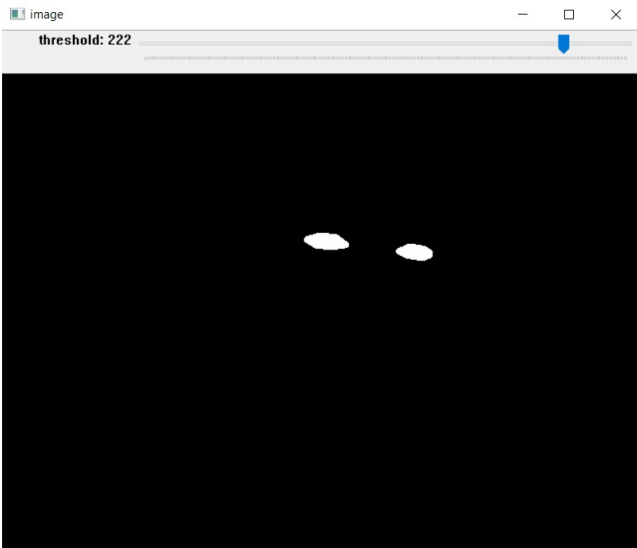


Figura 4: Binarização com limiar manual (222): apenas as regiões dos olhos permanecem.

cinco pontos), m o ponto médio entre os cantos interno e externo e



Figura 5: Centros dos olhos marcados em vermelho sobre a imagem da webcam.

w a distância entre esses cantos. A posição normalizada da íris é

$$e_x = \frac{c_x - m_x}{w}, \quad e_y = \frac{c_y - m_y}{w}, \quad (1)$$

calculada como média dos dois olhos. A divisão por w torna a medida independente da distância até a câmera. Para compensar parcialmente o movimento da cabeça, calcula-se também a posição da ponta do nariz n em relação ao ponto médio o entre os cantos externos dos olhos, normalizada pela distância interocular d : $(h_x, h_y) = (n - o)/d$. O vetor de características de cada amostra é $z = (e_x, e_y, h_x, h_y)$.

Piscadas são detectadas pela razão de aspecto do olho (*eye aspect ratio*, EAR) [33], aqui simplificada para a razão entre a abertura vertical e a largura do olho, $EAR = \|p_{sup} - p_{inf}\|/w$. Como a EAR natural varia entre pessoas, o limiar é calibrado individualmente em uma pré-checagem de alguns segundos: o participante primeiro mantém os olhos abertos e depois pisca naturalmente, e o limiar é posicionado entre a mediana da EAR com olhos abertos e o percentil 5 da fase de piscadas. Amostras com piscada, ou em que o rosto não é detectado, são marcadas como perdidas. Como alternativa, o sistema pode usar o detector de Viola-Jones [35], que não depende de redes neurais, mas não detecta piscadas.

3.3.2 Calibração por regressão. A calibração exhibe, em tela cheia e em ordem aleatória, nove alvos em uma grade de 3×3 (a 10%, 50% e 90% da largura e da altura). Cada alvo permanece por 2 s, e as amostras dos primeiros 0,7 s são descartadas para dar tempo ao olhar de se estabilizar. Em cada alvo, amostras atípicas são removidas

por um critério robusto (mais de três desvios absolutos medianos da mediana). As características são padronizadas e o mapeamento para a tela $\hat{s} = \mathbf{W}^T \phi(\mathbf{z})$ é ajustado por regressão *ridge*:

$$\min_{\mathbf{W}} \sum_i \|\mathbf{s}_i - \mathbf{W}^T \phi(\mathbf{z}_i)\|^2 + \lambda \|\mathbf{W}'\|^2, \quad (2)$$

em que $\mathbf{s}_i$ é a posição do alvo, $\mathbf{W}'$ exclui o intercepto e $\lambda = 10^{-2}$. Foram avaliadas duas expansões: a linear, $\phi(\mathbf{z}) = (1, e_x, e_y, h_x, h_y)$, e a polinomial de segunda ordem, $\phi(\mathbf{z}) = (1, e_x, e_y, e_x e_y, e_x^2, e_y^2, h_x, h_y)$.

3.3.3 Suavização e detecção de fixações. As estimativas são suavizadas pelo filtro One Euro [6], um passa-baixas cuja frequência de corte aumenta com a velocidade do sinal: ele reduz o tremor quando o olhar está parado e introduz pouco atraso nas sacadas. O filtro é reiniciado após lacunas maiores que 200 ms. As fixações são identificadas pelo algoritmo I-DT [31], que agrupa amostras consecutivas cuja dispersão $D = (\max x - \min x) + (\max y - \min y)$ não excede um limiar. Foram usados dispersão máxima de 120 px, duração mínima de 100 ms e lacuna máxima de 100 ms dentro de uma fixação. **[Justifique o limiar de 120 px em graus, com base na acurácia medida: ele deve ser maior que o ruído do sistema, senão uma única fixação é fragmentada em várias.]**

3.3.4 Validação e métricas de qualidade. Após a calibração, cinco alvos novos (quatro na grade de 30% e 70% e um no centro), que não participam do ajuste do modelo, são exibidos com o mesmo procedimento. Para cada alvo $\mathbf{t}$ com estimativas $\mathbf{p}_1, \dots, \mathbf{p}_n$, calculam-se [14] a acurácia $\|\bar{\mathbf{p}} - \mathbf{t}\|$, a precisão RMS entre amostras sucessivas,

$$\theta_{\text{RMS}} = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n-1} \|\mathbf{p}_{i+1} - \mathbf{p}_i\|^2}, \quad (3)$$

e a proporção de amostras perdidas. Os valores em pixels são convertidos para graus de ângulo visual a partir da largura física da tela L (em cm), da sua resolução horizontal R e da distância D_o entre o olho e a tela:

$$\text{px}/^\circ = \frac{R}{L} \cdot 2D_o \tan(0,5^\circ). \quad (4)$$

A validação é calculada com as estimativas brutas, sem suavização, para não mascarar a precisão real do sistema.

3.4 Protocolo experimental

[Preencher após realizar o experimento. Sugestão: 5 a 10 participantes já tornam o trabalho defensável.]

Participantes. Participaram $[N]$ voluntários ([idade média, quantos usam óculos]). Nenhum vídeo é armazenado: o sistema grava apenas as características numéricas extraídas, o que reduz riscos à privacidade dos participantes. **[Mencione o termo de consentimento, se houver.]**

Equipamento. [Modelo do notebook ou computador, webcam e resolução da câmera, monitor (polegadas, resolução e largura física), distância aproximada de 60 cm, condições de iluminação.]

Estímulo e tarefa. [Sugestão que conversa com a Figura 1: um print da página inicial de um site com a tarefa “encontre onde localizar uma loja”, exibido por 30 s. Defina as AOIs (menu, banner, busca, localizador etc.).]

Tabela 1: Qualidade dos dados por participante (modelo polinomial com pose da cabeça). Acurácia e precisão medidas nos alvos de validação.

Part.	Acurácia (°)	Precisão (°)	Perda val. (%)	Perda grav. (%)
P01	[-]	[-]	[-]	[-]
P02	[-]	[-]	[-]	[-]
...				
Média (DP)	[-]	[-]	[-]	[-]

Procedimento. Cada sessão segue a ordem: pré-checagem de detecção e piscada (6 s), calibração (9 alvos), validação (5 alvos) e exibição do estímulo com a tarefa. A sessão completa leva cerca de [2] minutos.

Análise. Para cada participante são calculadas as métricas de qualidade da validação e da gravação. Os modelos de calibração (linear e polinomial, com e sem as características da cabeça) são comparados *offline*: cada modelo é treinado apenas com as amostras dos alvos de calibração e avaliado nos alvos de validação. Por fim, as fixações de todos os participantes são agregadas em um mapa de calor, e calculam-se, por AOI, o TTFF, o número de fixações, o tempo de permanência e o número de visitas.

3.5 Tecnologias utilizadas

O sistema foi implementado em Python, com OpenCV¹ [4] para captura e processamento de imagens, NumPy² [10] para a regressão e o processamento numérico, e MediaPipe³ [20] para o Face Mesh. O protótipo inicial usou a dlib⁴ [18]. O repositório inclui testes automatizados, entre eles uma simulação completa da sessão com câmera e rosto sintéticos, usada para verificar o *pipeline* antes dos experimentos com pessoas.

4 RESULTADOS E DISCUSSÃO

[Os valores desta seção saem diretamente dos arquivos gerados por `analyze_session.py` (`summary.csv`, `model_comparison.csv`, `aoi_metrics.csv`).]

4.1 Qualidade dos dados

A Tabela 1 resume a qualidade dos dados de cada participante.

[Comente: a acurácia média é compatível com o que a literatura relata para webcam? Algum participante destoa (óculos, iluminação)? Lembre que rastreadores infravermelhos ficam abaixo de 1°. Compare com os números reportados nos trabalhos relacionados, conferindo-os nos artigos originais.]

4.2 Comparação dos modelos de calibração

[Discuta a Tabela 2. O modelo polinomial tem mais parâmetros: ele generaliza melhor para os alvos de validação ou sobreajusta os 9 pontos? A pose da cabeça ajuda? Qualquer

¹<https://opencv.org/>

²<https://numpy.org/>

³<https://ai.google.dev/edge/mediapipe>

⁴<http://dlib.net/>

Tabela 2: Acurácia média na validação (graus) por modelo de calibração, treinado apenas com os alvos de calibração.

Modelo	Com pose da cabeça	Sem pose da cabeça
Linear	[-]	[-]
Polinomial (2ª ordem)	[-]	[-]

[Inserir group_heatmap.png gerado pelo
analyze_session.py]

Figura 6: Mapa de calor agregado das fixações dos [N] participantes durante a tarefa [descrever a tarefa].

Tabela 3: Métricas por área de interesse (média entre participantes).

AOI	Encontraram (%)	TTF (s)	Fixações	Permanência (s)
[menu]	[-]	[-]	[-]	[-]
[busca]	[-]	[-]	[-]	[-]

resultado é válido e interessante, desde que seja o resultado real.]

4.3 Aplicação em experiência do usuário

[Interprete a Figura 6 e a Tabela 3 como um pesquisador de UX faria: o elemento-alvo foi encontrado? Em quanto tempo? Que região atraiu atenção sem ser útil para a tarefa? Relacione com Norman (visibilidade, affordances) e com a Figura 1.]

4.4 Limitações e ameaças à validade

A pose da cabeça é estimada de forma aproximada, a partir de pontos 2D; movimentos amplos após a calibração degradam o mapeamento, o que exigiria recalibração ou apoio de queixo. Óculos com reflexo, pouca iluminação e contraluz aumentam a perda de dados. A taxa de amostragem é limitada pela webcam (tipicamente 30 Hz), insuficiente para medir sacadas com precisão, de modo que o trabalho se restringe a métricas baseadas em fixações. Por fim, a amostra de participantes é pequena e não representativa, e os resultados devem ser lidos como uma avaliação de viabilidade, não como uma validação definitiva. [Acrescente limitações observadas na prática.]

5 CONSIDERAÇÕES FINAIS

Este trabalho desenvolveu um protótipo de rastreamento ocular baseado apenas em uma webcam e em técnicas de Inteligência Artificial, com o objetivo de obter dados úteis para avaliar a experiência do usuário. A evolução de um primeiro protótipo, baseado em HOG e em árvores de regressão, para um *pipeline* com uma CNN de detecção da íris, calibração por regressão regularizada, filtragem adaptativa e detecção de fixações permitiu passar da simples localização dos olhos para a estimativa do ponto observado na tela e para métricas de UX.

[Retome cada objetivo específico e diga se foi atingido, com os números principais: acurácia, precisão, perda de dados, melhor modelo de calibração e o principal achado de UX.]

[Conclusão honesta sobre a viabilidade. Por exemplo: a acurácia obtida permite identificar quais regiões de uma página recebem atenção, desde que as AOIs sejam grandes em relação ao erro, mas não substitui um rastreador dedicado em análises finas, como a leitura.]

Como trabalhos futuros, destacam-se: substituir o mapeamento por características por modelos baseados em aparência com normalização da pose da cabeça [37, 28]; recalibrar o modelo continuamente a partir das interações do usuário, como no WebGazer [26]; avaliar o uso de apenas um olho quando ele for mais confiável [15]; e portar o sistema para o navegador, permitindo testes de UX remotos com um número maior de participantes.

REFERÊNCIAS

- [1] Artsiom Ablavatski, Andrey Vakunov, Ivan Grishchenko, Karthik Raveendran e Matsvei Zhdanovich. 2020. Real-time pupil tracking from monocular video for digital puppetry. (2020). arXiv: 2006.11341.
- [2] R. Andersson, L. Larsson, K. Holmqvist, M. Stridh e M. Nyström. 2017. One algorithm to rule them all? an evaluation and discussion of ten eye movement event-detection algorithms. *Behavior Research Methods*, 49, 616–637. doi: 10.3758/s13428-016-0738-9.
- [3] Agnieszka Bojko. 2005. Eye tracking in user experience testing: how to make the most of it. Em *Proceedings of the 14th Annual Conference of the Usability Professionals' Association (UPA 2005)*. Montréal, Canada.
- [4] Gary Bradski. 2000. The OpenCV library. *Dr. Dobbs' Journal of Software Tools*.
- [5] C. Braunagel, D. Geisler, W. Stolzmann, W. Rosenstiel e E. Kasneci. 2016. On the necessity of adaptive eye movement classification in conditionally automated driving scenarios. Em *Proceedings of the Symposium on Eye Tracking Research and Applications (ETRA '16)*. ACM, 19–26. doi: 10.1145/2857491.2857529.
- [6] Géry Casiez, Nicolas Roussel e Daniel Vogel. 2012. 1€ filter: a simple speed-based low-pass filter for noisy input in interactive systems. Em *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '12)*. ACM, 2527–2530. doi: 10.1145/2207676.2208639.
- [7] Navneet Dalal e Bill Triggs. 2005. Histograms of oriented gradients for human detection. Em *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*. Vol. 1, 886–893. doi: 10.1109/CVPR.2005.177.
- [8] Andrew T. Duchowski. 2017. *Eye Tracking Methodology: Theory and Practice*. (3ª ed.). Springer, Cham. doi: 10.1007/978-3-319-57883-5.
- [9] W. Fuhl, Y. Rong e E. Kasneci. 2021. Fully convolutional neural networks for raw eye tracking data segmentation, generation, and reconstruction. Em *25th International Conference on Pattern Recognition (ICPR)*, 142–149. doi: 10.1109/ICPR48806.2021.9413268.
- [10] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt et al. 2020. Array programming with NumPy. *Nature*, 585, 357–362. doi: 10.1038/s41586-020-2649-2.
- [11] R. S. Hessels, R. Andersson, I. T. C. Hooge, M. Nyström e C. Kemner. 2015. Consequences of eye color, positioning, and head movement for eye-tracking data quality in infant research. *Infancy*, 20, 6, 601–633. doi: 10.1111/infa.12093.
- [12] Roy S. Hessels e Ignace T. C. Hooge. 2019. Eye tracking in developmental cognitive neuroscience – the good, the bad and the ugly. *Developmental Cognitive Neuroscience*, 40, 100710. doi: 10.1016/j.dcn.2019.100710.
- [13] Kenneth Holmqvist, Marcus Nyström, Richard Andersson, Richard Dewhurst, Halszka Jarodzka e Joost van de Weijer. 2011. *Eye Tracking: A Comprehensive Guide to Methods and Measures*. Oxford University Press, Oxford.

- [14] Kenneth Holmqvist, Marcus Nyström e Fiona Mulvey. 2012. Eye tracker data quality: what it is and how to measure it. Em *Proceedings of the Symposium on Eye Tracking Research and Applications (ETRA '12)*. ACM, New York, NY, USA, 45–52. doi: 10.1145/2168556.2168563.
- [15] I. T. C. Hooge, G. A. Holleman, N. C. Haukes e R. S. Hessels. 2019. Gaze tracking accuracy in humans: one eye is sometimes better than two. *Behavior Research Methods*, 51, 2712–2721. doi: 10.3758/s13428-018-1135-3.
- [16] Yury Kartynnik, Artsiom Ablavatski, Ivan Grishchenko e Matthias Grundmann. 2019. Real-time facial surface geometry from monocular video on mobile GPUs. CVPR Workshop on Computer Vision for Augmented and Virtual Reality. (2019). arXiv: 1907.06724.
- [17] Vahid Kazemi e Josephine Sullivan. 2014. One millisecond face alignment with an ensemble of regression trees. Em *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1867–1874. doi: 10.1109/CVPR.2014.241.
- [18] Davis E. King. 2009. Dlib-ml: a machine learning toolkit. *Journal of Machine Learning Research*, 10, 1755–1758.
- [19] Kyle Krafka, Aditya Khosla, Petr Kellnhofer, Harini Kannan, Suchendra Bhandarkar, Wojciech Matusik e Antonio Torralba. 2016. Eye tracking for everyone. Em *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2176–2184. doi: 10.1109/CVPR.2016.239.
- [20] Camillo Lugaresi et al. 2019. Mediapipe: a framework for building perception pipelines. (2019). arXiv: 1906.08172.
- [21] N. Modi e J. Singh. 2022. Real-time camera-based eye gaze tracking using convolutional neural network: a case study on social media website. *Virtual Reality*, 26, 1489–1506. doi: 10.1007/s10055-022-00642-6.
- [22] Peter Morville. 2004. User experience design. Semantic Studios. Retrieved 2 de out. de 2026 from https://semanticstudios.com/user_experience_design/.
- [23] Jakob Nielsen. 1993. *Usability Engineering*. Morgan Kaufmann, San Francisco, CA.
- [24] Donald A. Norman. 2002. *The Design of Everyday Things*. Basic Books, New York, NY.
- [25] M. Nyström, R. Andersson, K. Holmqvist e J. van de Weijer. 2013. The influence of calibration method and eye physiology on eyetracking data quality. *Behavior Research Methods*, 45, 1, 272–288. doi: 10.3758/s13428-012-0247-4.
- [26] Alexandra Papoutsaki, Patsorn Sangkloy, James Laskey, Nediya Daskalova, Jeff Huang e James Hays. 2016. Webgazer: scalable webcam eye tracking using user interactions. Em *Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI)*, 3839–3845.
- [27] Seonwook Park, Emre Aksan, Xucong Zhang e Otmar Hilliges. 2020. Towards end-to-end video-based eye-tracking. Em *European Conference on Computer Vision (ECCV)*.
- [28] Seonwook Park, Xucong Zhang, Andreas Bulling e Otmar Hilliges. 2018. Learning to find eye region landmarks for remote gaze estimation in unconstrained settings. Em *Proceedings of the ACM Symposium on Eye Tracking Research and Applications (ETRA '18)*. ACM, Warsaw, Poland.
- [29] M. C. Pretorius, J. van Biljon e E. de Kock. 2010. Added value of eye tracking in usability studies: expert and non-expert participants. Em *Human-Computer Interaction (HCIS 2010)* (IFIP Advances in Information and Communication Technology). P. Forbrig, F. Paternó e A. Mark Pejtersen, editores. Vol. 332. Springer, Berlin, Heidelberg. doi: 10.1007/978-3-642-15231-3_12.
- [30] Christos Sagonas, Georgios Tzimiropoulos, Stefanos Zafeiriou e Maja Pantic. 2013. 300 faces in-the-wild challenge: the first facial landmark localization challenge. Em *IEEE International Conference on Computer Vision Workshops (ICCVW)*, 397–403.
- [31] Dario D. Salvucci e Joseph H. Goldberg. 2000. Identifying fixations and saccades in eye-tracking protocols. Em *Proceedings of the Symposium on Eye Tracking Research and Applications (ETRA '00)*. ACM, New York, NY, USA, 71–78. doi: 10.1145/355017.355028.
- [32] Irwin Sobel e Gary Feldman. 1973. A 3x3 isotropic gradient operator for image processing. Em *Pattern Classification and Scene Analysis*. Richard O. Duda e Peter E. Hart, editores. Apresentado originalmente no Stanford Artificial Intelligence Project, 1968. John Wiley & Sons, New York, 271–272.
- [33] Tereza Soukupová e Jan Čech. 2016. Real-time eye blink detection using facial landmarks. Em *21st Computer Vision Winter Workshop (CVWW)*. Rimske Toplice, Slovenia.
- [34] J. Strohl, C. Gonzalez, J. Sauser, S. Montazeri e B. Griepentrog. 2015. Creating forms and disclosures that work: using eye tracking to improve the user experience. Em *Universal Access in Human-Computer Interaction. Access to Today's Technologies (UAHCI 2015)* (Lecture Notes in Computer Science). M. Antonia e C. Stephanidis, editores. Vol. 9175. Springer, Cham. doi: 10.1007/978-3-319-20678-3_12.
- [35] Paul Viola e Michael Jones. 2001. Rapid object detection using a boosted cascade of simple features. Em *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2001)*. Vol. 1, 1-511–1-518. doi: 10.1109/CVPR.2001.990517.
- [36] B. A. Zardari, Z. Hussain, A. A. Arain, W. H. Rizvi e M. S. Vighio. 2021. QUEST e-learning portal: applying heuristic evaluation, usability testing and eye tracking. *Universal Access in the Information Society*, 20, 3, 531–543. doi: 10.1007/s10209-020-00774-z.
- [37] Xucong Zhang, Yusuke Sugano e Andreas Bulling. 2018. Revisiting data normalization for appearance-based gaze estimation. Em *Proceedings of the ACM Symposium on Eye Tracking Research and Applications (ETRA '18)*. ACM, 12:1–12:9.